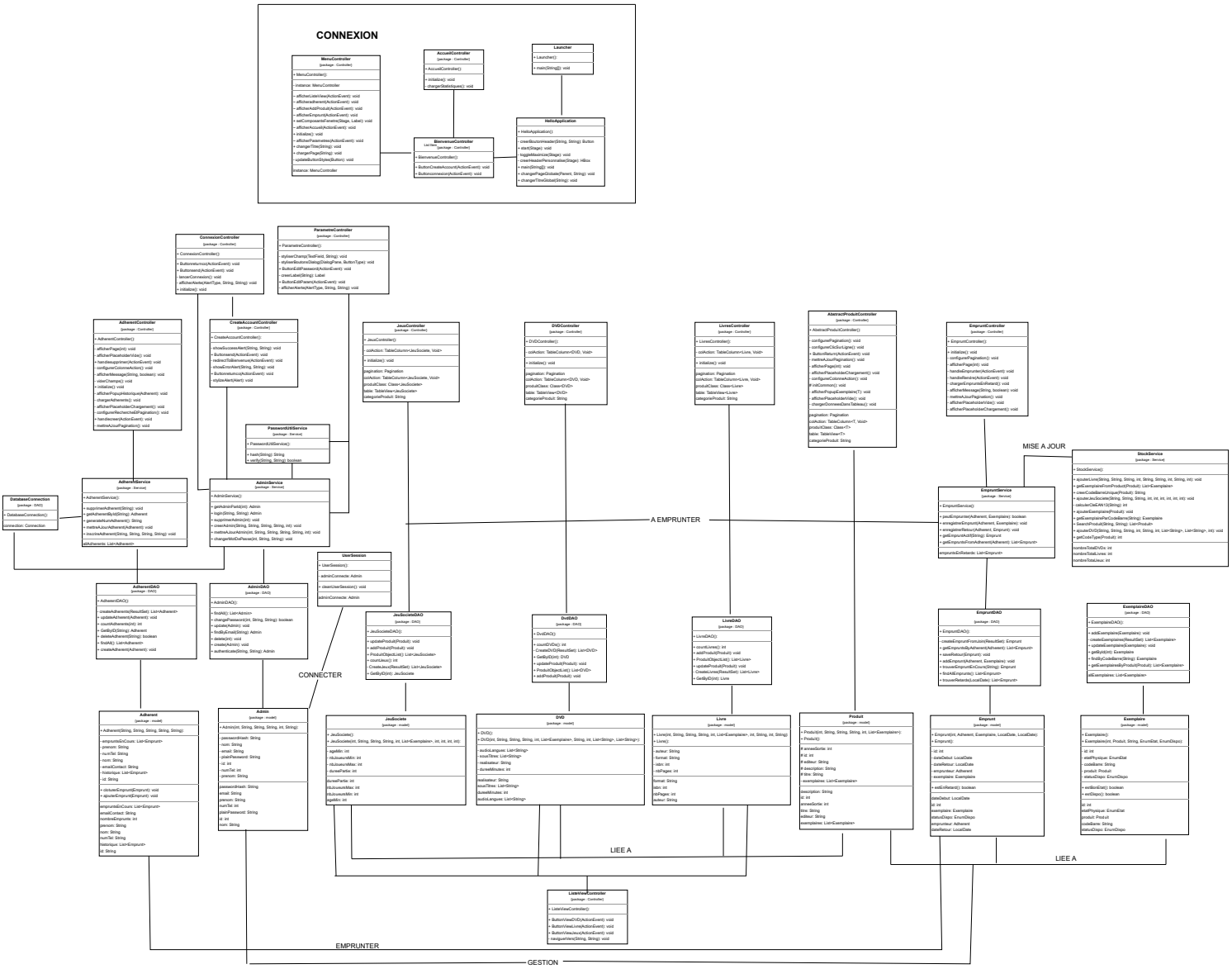
CONNEXION
MenuController
(package : Controller)
+ MenuController()
- instance: MenuController
+ afficherListeView(ActionEvent): void
+ afficherAdherent(ActionEvent): void
+ afficherAddProduit(ActionEvent): void
+ afficherEmprunt(ActionEvent): void
+ setComposantsFenetre(Stage, Label): void
+ afficherAccueil(ActionEvent): void
+ initialize(): void
+ afficherParametres(ActionEvent): void
+ changerTitre(String): void
+ changerPage(String): void
- updateButtonStyles(Button): void
+ instance: MenuController
AccueilController
(package : Controller)
+ AccueilController()
+ initialize(): void
- chargerStatistiques(): void
Launcher
+ Launcher()
+ main(String[]): void
BienvenueController
List Stage
(package : Controller)
+ BienvenueController()
+ ButtonCreateAccount(ActionEvent): void
+ Buttonconnexion(ActionEvent): void
HelloApplication
+ HelloApplication()
- creerBouttonHeader(String, String): Button
+ start(Stage): void
- toggleMaximize(Stage): void
- creerHeaderPersonnalise(Stage): HBox
+ main(String[]): void
+ changerPageGotoIdParent(Stage, String): void
+ changerTitreGlobal(String): void
ConnexionController
(package : Controller)
+ ConnexionController()
+ Buttonreturncx(ActionEvent): void
+ Buttonsend(ActionEvent): void
- lancerConnexion(): void
- afficherAlerte(AlertType, String, String): void
+ initialize(): void
ParametreController
(package : Controller)
+ ParametreController()
- stylerChampTextField, String): void
- stylerBouttonDialog(DialogPane, ButtonType): void
+ ButtonEditPassword(ActionEvent): void
- creerLabel(String): Label
+ ButtonEditParam(ActionEvent): void
- afficherAlerte(AlertType, String, String): void
AdherentController
(package : Controller)
+ AdherentController()
- afficherPage(int): void
- afficherPlaceholderVide(): void
+ handleupprimer(ActionEvent): void
- configurerColonneAction(): void
- afficherMessage(String, boolean): void
- initerChamps(): void
+ initialize(): void
- afficherPopupUpdate(Adherent): void
- chargerAdherents(): void
- afficherPlaceholderChargement(): void
- configurerRechercheEtPagination(): void
+ handlecreer(ActionEvent): void
- mettreAJourPagination(): void
CreateAccountController
(package : Controller)
+ CreateAccountController()
- showSuccessAlert(String, String): void
+ Buttonsend(ActionEvent): void
- redirectToBienvenue(ActionEvent): void
- showErrorAlert(String, String): void
+ Buttonreturncx(ActionEvent): void
- stylerAlert(Alert): void
JeuxController
(package : Controller)
+ JeuxController()
- colAction: TableColumn<JeuSociete, Void>
+ initialize(): void
pagination: Pagination
colAction: TableColumn<JeuSociete, Void>
produitClass: Class<JeuSociete>
table: TableView<JeuSociete>
categorieProduit: String
DVDController
(package : Controller)
+ DVDController()
- colAction: TableColumn<DVD, Void>
+ initialize(): void
pagination: Pagination
colAction: TableColumn<DVD, Void>
produitClass: Class<DVD>
table: TableView<DVD>
categorieProduit: String
LivreController
(package : Controller)
+ LivreController()
- colAction: TableColumn<Livre, Void>
+ initialize(): void
pagination: Pagination
colAction: TableColumn<Livre, Void>
produitClass: Class<Livre>
table: TableView<Livre>
categorieProduit: String
AbstractProduitController
(package : Controller)
+ AbstractProduitController()
- configurePagination(): void
- configurerSlideLigne(): void
+ ButtonReturn(ActionEvent): void
- mettreAJourPagination(): void
- afficherPage(int): void
- afficherPlaceholderChargement(): void
- configurerColonneAction(): void
initCommon(): void
- afficherPopupExemplaire(T): void
- afficherPlaceholderVide(): void
- chargerDonneesDansTableau(): void
pagination: Pagination
colAction: TableColumn<T, Void>
produitClass: Class<T>
table: TableView<T>
categorieProduit: String
EmpruntController
(package : Controller)
+ EmpruntController()
+ initialize(): void
- configurePagination(): void
- afficherPage(int): void
- handleEmprunter(ActionEvent): void
- handleRetourn(ActionEvent): void
- chargerEmpruntsEnRetard(): void
- afficherMessage(String, boolean): void
- mettreAJourPagination(): void
- afficherPlaceholderVide(): void
- afficherPlaceholderChargement(): void
MISE A JOUR
StockService
(package : Service)
+ StockService()
+ ajouterLivre(String, String, String, int, String, String, int, String, int): void
+ getExemplaireFromProduit(Produit): List<Exemplaire>
+ creerCodeBarreUniqueProduit(): String
+ ajouterJeuSociete(String, String, String, int, int, int, int, int, int): void
- calculerCleEAN13(String): int
+ ajouterExemplaireProduit(): void
+ getExemplaireParCodeBarre(String): Exemplaire
+ SearchProduit(String, String): List<Produit>
+ ajouterDVD(String, String, String, int, String, int, List<String>, List<String>, int): void
+ getCodeType(Produit): int
nombreTotalDVDs: int
nombreTotalLivre: int
nombreTotalJeux: int
PasswordUtilService
(package : Service)
+ PasswordUtilService()
+ hash(String): String
+ verify(String, String): boolean
DatabaseConnection
(package : DAO)
+ DatabaseConnection()
connection: Connection
AdherentService
(package : Service)
+ AdherentService()
+ supprimerAdherent(String): void
+ getAdherentById(String): Adherent
+ genererMotJourAdherent(): String
+ mettreAJourAdherent(Adherent): void
+ InscrireAdherent(String, String, String, String): void
- sHAdherents: List<Adherent>
AdminService
(package : Service)
+ AdminService()
- getAdminFromId(int): Admin
+ login(String, String): Admin
+ supprimerAdmin(int): void
+ creerAdmin(String, String, String, String, int): void
+ mettreAJourAdmin(int, String, String, String, String, int): void
+ changerMotDePasse(int, String, String): void
EmpruntService
(package : Service)
+ EmpruntService()
+ peutEmprunter(Adherent, Exemplaire): boolean
+ enregistrerEmprunt(Adherent, Exemplaire): void
+ enregistrerRetour(Adherent, Emprunt): void
+ getEmpruntsActifS(String): Emprunt
+ getEmpruntsFromAdherent(Adherent): List<Emprunt>
empruntsEnRetards: List<Emprunt>
A EMPRUNTER
UserSession
+ UserSession()
- adminConnecte: Admin
+ cleanUserSession(): void
adminConnecte: Admin
AdherentDAO
(package : DAO)
+ AdherentDAO()
- createAdherents(ResultSet): List<Adherent>
+ updateAdherent(Adherent): void
+ countAdherents(int): int
- GetByID(String): Adherent
+ deleteAdherent(String): boolean
+ findAll(): List<Adherent>
- createAdherent(Adherent): void
AdminDAO
(package : DAO)
+ AdminDAO()
+ findAll(): List<Admin>
+ changePassword(int, String, String): boolean
+ updateAdmin(): void
+ findByEmail(String): Admin
+ delete(int): void
+ createAdmin(): void
+ authenticate(String, String): Admin
CONNECTER
JeuSocieteDAO
(package : DAO)
+ JeuSocieteDAO()
+ updateProduit(Produit): void
+ addProduit(Produit): void
+ ProduitObjectList(): List<JeuSociete>
+ countJeux(): int
- CreateJeux(ResultSet): List<JeuSociete>
+ GetByID(int): JeuSociete
DvdDAO
(package : DAO)
+ DvdDAO()
+ countDVDs(): int
- CreateDVD(ResultSet): List<DVD>
+ GetByID(int): DVD
+ updateProduit(Produit): void
+ ProduitObjectList(): List<DVD>
+ addProduit(Produit): void
LivreDAO
(package : DAO)
+ LivreDAO()
+ countLivres(): int
+ addProduit(Produit): void
+ ProduitObjectList(): List<Livre>
+ updateProduit(Produit): void
- CreateLivres(ResultSet): List<Livre>
+ GetByID(int): Livre
EmpruntDAO
(package : DAO)
+ EmpruntDAO()
- createEmpruntFromJoin(ResultSet): Emprunt
+ getEmpruntsByAdherent(Adherent): List<Emprunt>
+ saveRetour(Emprunt): void
+ addEmprunt(Adherent, Exemplaire): void
+ trouverEmpruntEnCours(String): Emprunt
+ findAllEmprunts(): List<Emprunt>
+ trouverRetardsLocalDate(): List<Emprunt>
ExemplaireDAO
(package : DAO)
+ ExemplaireDAO()
+ addExemplaire(Exemplaire): void
- createExemplaires(ResultSet): List<Exemplaire>
+ updateExemplaire(Exemplaire): void
+ getById(int): Exemplaire
+ findByCodeBarre(String): Exemplaire
+ getExemplairesByProduit(Produit): List<Exemplaire>
allExemplaires: List<Exemplaire>
Adherent
(package : model)
+ Adherent(String, String, String, String, String):
- empruntsEnCours: List<Emprunt>
- prenom: String
- numTel: String
- nom: String
- emailContact: String
- historique: List<Emprunt>
- id: String
+ ajouterEmprunt(Emprunt): void
+ ajouterEmprunt(Emprunt): void
empruntsEnCours: List<Emprunt>
emailContact: String
nombreEmprunts: int
prenom: String
nom: String
numTel: String
historique: List<Emprunt>
id: String
Admin
(package : model)
+ Admin(int, String, String, String, int, String):
- passwordHash: String
- nom: String
- email: String
- plainPassword: String
- id: int
- numTel: int
- prenom: String
passwordHash: String
email: String
prenom: String
numTel: int
plainPassword: String
id: int
nom: String
JeuSociete
(package : model)
+ JeuSociete():
+ JeuSociete(int, String, String, String, int, List<Exemplaire>, int, int, int, int):
- ageMin: int
- nbJouueurMin: int
- nom: String
- dureePartie: int
dureePartie: int
nbJouueurMax: int
nbJouueurMin: int
ageMin: int
DVD
(package : model)
+ DVD():
+ DVD(int, String, String, String, int, List<Exemplaire>, String, int, List<String>, List<String>):
- audioLangues: List<String>
- sousTitres: List<String>
- realisateur: String
- dureeMinutes: int
realisateur: String
sousTitres: List<String>
dureeMinutes: int
audioLangues: List<String>
Livre
(package : model)
+ Livre(int, String, String, String, int, List<Exemplaire>, int, String, int, String):
+ Livre():
- auteur: String
- format: String
- isbn: int
- nbPages: int
format: String
isbn: int
nbPages: int
auteur: String
Produit
(package : model)
+ Produit(int, String, String, String, int, List<Exemplaire>):
+ Produit():
# anneeSortie: int
# id: int
# editeur: String
# description: String
# titre: String
- exemplaires: List<Exemplaire>
description: String
id: int
anneeSortie: int
titre: String
editeur: String
exemplaires: List<Exemplaire>
Emprunt
(package : model)
+ Emprunt(int, Adherent, Exemplaire, LocalDate, LocalDate):
+ Emprunt():
- id: int
- dateDebut: LocalDate
- dateRetour: LocalDate
- emprunteur: Adherent
- exemplaire: Exemplaire
+ estEnRetard(): boolean
dateDebut: LocalDate
id: int
exemplaire: Exemplaire
statusDispo: EnumDispo
emprunteur: Adherent
dateRetour: LocalDate
Exemplaire
(package : model)
+ Exemplaire():
+ Exemplaire(int, Produit, String, EnumEtat, EnumDispo):
- id: int
- etatPhysique: EnumEtat
- codeBarre: String
- produit: Produit
- statusDispo: EnumDispo
+ setStorEtat(): boolean
+ estDispo(): boolean
id: int
etatPhysique: EnumEtat
produit: Produit
codeBarre: String
statusDispo: EnumDispo
LIEE A
LIEE A
ListeViewController
(package : Controller)
+ ListeViewController():
+ ButtonViewDVD(ActionEvent): void
+ ButtonViewLivre(ActionEvent): void
+ ButtonViewJeux(ActionEvent): void
- navigueVers(String, String): void
EMPRUNTER
GESTION